

PARADIGMAS DE PROGRAMAÇÃO

Exame de Preparação 3 — Reestruturado (Modelo 13,5 / 6,5)

UC: Paradigmas de Programação (PP)

Ano Letivo: 2025/2026

Duração: 2 horas

Material: Exame sem consulta

Parte I — Aplicação Prática (Conceito do Trabalho)

13,5 valores

1. Passagem de Parâmetros e Referências em Java (3,5 valores)

Considere a seguinte classe utilitária de testes do teu projeto:

```
public class RouteManager {  
    public void resetVehicle(Vehicle v) {  
        v = null;  
    }  
  
    public static void main(String[] args) {  
        RouteManager rm = new RouteManager();  
        Vehicle vehicle = new VehicleImpl(ItemType.MEDICINE, 400.0);  
        rm.resetVehicle(vehicle);  
        System.out.println(vehicle.getSupplyType());  
    }  
}
```

1. Qual é o output exato gerado na consola pela execução do método main?
2. Explique a razão física na memória JVM para a alteração feita em `v = null;` dentro do método `resetVehicle` não afetar a variável `vehicle` no método chamador `main`.

2. Implementação de Interfaces e Lógica de Ocupação de Contentores (3,5 valores)

Considere a interface `Telemetry` abaixo declarada no projeto:

```
public interface Telemetry {  
    public String getOccupationStatus();  
}
```

Implemente a interface `Telemetry` na classe `ContainerImpl` de forma a retornar uma string formatada contendo o código do contentor e a sua percentagem de ocupação atual.

Nota: A percentagem de ocupação é obtida dividindo o valor da medição mais recente do contentor (da lista `getMeasurements()`) pela capacidade máxima do contentor. Caso não existam medições, assume-se ocupação de 0.0%.

```
public class ContainerImpl implements Container, Telemetry {  
    private final String code;  
    private final double capacity;  
  
    // ... construtores e métodos existentes ...  
  
    @Override  
    public String getOccupationStatus() {  
        // TODO: Implemente a sua resposta aqui  
    }  
}
```

3. Gestão Logística, Exceções Personalizadas e Redimensionamento de Arrays (6,5 valores)

Crie uma classe chamada `GestaoVeiculos` para gerir a frota da instituição. **Não utilize nenhuma classe do Java Collections Framework (JCF).**

A classe deve gerir um vetor interno de veículos (`Vehicle[]`) e conter:

1. O vetor de armazenamento e o respetivo contador físico de veículos.
2. O construtor que inicializa o vetor com capacidade física inicial de 2 posições.
3. O método `public boolean registarVeiculo(Vehicle v)`: adiciona o veículo ao vetor. Caso esteja cheio, duplica o tamanho do vetor manualmente. Retorna `false` se for tentada a adição de um veículo nulo ou que já exista no vetor (comparando por referência). Retorna `true` se for inserido com sucesso.
4. O método `public double obterCapacidadeTotal(ItemType tipo) throws VehicleNotFoundException`: calcula e retorna a capacidade máxima total acumulada de transporte para o tipo de bem `tipo`. Se não existirem veículos registados para esse tipo de bem na frota, o método deve lançar a exceção **checked** `VehicleNotFoundException`.

```
public class GestaoVeiculos {  
    // TODO: Implemente os atributos, construtor, registarVeiculo e obterCapacidadeTotal  
}
```

Parte II — Conceitos Teóricos

6,5 valores

4. Perguntas de Escolha Múltipla (0,5 val. cada) (3,5 valores)

Selecione a única opção correta para cada uma das alíneas:

1. A sobrecarga (overloading) de construtores em Java consiste em:
 - i. Substituir a lógica de inicialização de uma classe base numa subclasse.
 - ii. Declarar vários construtores na mesma classe com o mesmo nome mas com parâmetros de tipos ou quantidade diferentes.
 - iii. Inicializar as variáveis com valores por defeito da JVM sem construtores explícitos.
 - iv. Nenhuma das anteriores.
2. Se A e E forem classes, e B e D forem interfaces em Java, qual das seguintes declarações é válida?
 - i. `interface F implements B, D {}`
 - ii. `interface F extends E {}`
 - iii. `interface F extends B, D {}`

iv. `class F extends B {}`

3. Qual das seguintes é uma afirmação correta sobre o modificador `static` em Java?

- i. Os métodos estáticos podem aceder diretamente a atributos de instância não estáticos.
- ii. Pertence exclusivamente a instâncias específicas do objeto.
- iii. Os métodos estáticos pertencem à classe e podem ser invocados sem instanciar qualquer objeto da classe.
- iv. Lança uma exceção se a classe base for abstrata.

4. Uma exceção em Java do tipo **checked** (como `Exception`):

- i. Não é obrigatório efetuar o seu tratamento ou declaração na assinatura do método (`throws`).
- ii. O compilador obriga a que seja capturada num bloco `try-catch` ou propagada na assinatura com `throws`.
- iii. Lança sempre um erro fatal que termina a máquina virtual JVM imediatamente.
- iv. Deriva diretamente da classe `RuntimeException`.

5. Dois contentores `c1` (do tipo `ContainerImpl`) e `c2` (do tipo `RefrigeratedContainerImpl`) têm o mesmo código. O método `equals` está na classe base `ContainerImpl`. O que retorna `c1.equals(c2)` se utilizarmos o operador `getClass() != obj.getClass()` na validação?

- i. `true`
- ii. `false`
- iii. Lança um `ClassCastException`.
- iv. Nenhuma das anteriores.

6. A palavra-chave `extends` é utilizada em Java para:

- i. Permitir que uma classe implemente uma interface.
- ii. Permitir que uma classe herde de outra classe base ou que uma interface herde de outra(s) interface(s).
- iii. Declarar atributos imutáveis em tempo de compilação.
- iv. Nenhuma das anteriores.

7. Qual é a utilidade da palavra-chave `super` num construtor de uma subclasse em Java?

- i. Impedir o polimorfismo.
- ii. Chamar explicitamente um construtor compatível da classe base para inicializar os atributos herdados.
- iii. Limpar a memória garbage collector.
- iv. Nenhuma das anteriores.

5. Herança vs Composição e Ligação Dinâmica (3,0 valores)

No teu projeto, optou-se pela utilização de herança em `RefrigeratedVehicleImpl` (estendendo `VehicleImpl`). Explique qual é a principal vantagem desta decisão (herança) sobre a composição (criar um atributo de tipo `Vehicle` dentro da classe de refrigeração) para este caso, focando na **reutilização de código** e no **polimorfismo** (capacidade de passar veículos normais e refrigerados para o validador de rotas).